

Izračunavanje i privatnost

Privacy-preserving computation (PPC)

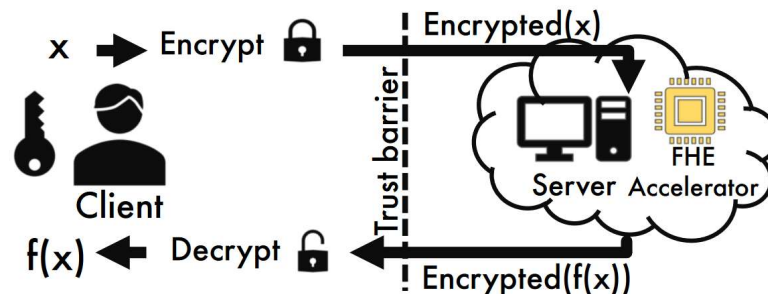
Tehnike PPC-ja [1]

- Homomorfna enkripcija (Homomorphic encryption)
- Tajno deljenje (Secret-sharing)
- Iskrivljenja kola (Garbled circuits)

	Conf	Cntrl	Arb	Sec	Overhead	Parties	Alone
HE	Yes	No	No	Noise	Very High	1	Yes
TFHE	Yes	No	Yes	Noise	Ext. High	1	Yes
SS	Yes	Yes	No	I.T.	Moderate	2(+)	No
GCs	Yes	Yes	Yes	AES	Very High	2	Yes

Homomorphic encryption

- Radi kao standardna enkripcija u mogućnost izračunavanja
 - end-to-end privatnost
- Idealno za računanje u oblaku



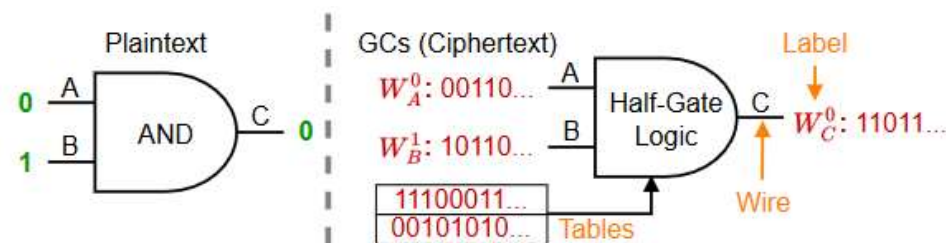
- Bazira se na šumu, koji raste sa dužinom izračunavanja
 - Na kraju rezultat može da se ne dekriptuje
- Celebrojno množenje i sabiranjem ili Bulov

Secret-sharing

- Podaci se dele učesnicima
 - Na osnovu dela ne može se konstruisati tajna informacija
- Svaki učesnik računa funkciju nad svojim delom
- Rezultati se kombinuju da se otkrije izlaz
- Podržane operacije su sabiranje i množenje
- Koristi i druge PPC-ijeve
- Dobro performanse za privatne neuralne mreže

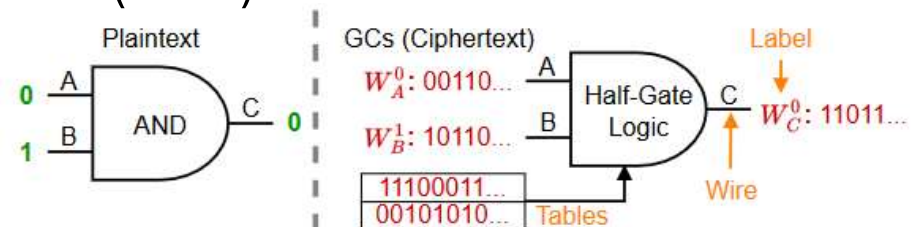
Garbled circuits

- Bulova algebra
 - Računanje proizvoljne funkcije
- Podaci i funkcija se enkriptuju
- Potrebna dva učesnika
- Veoma zahtevne operacije sa velikom količinom podataka



GC osnovna ideja

- A i B računaju funkciju $y = f(a, b)$
 - A poznaje a, B poznaje b
- Operandi se zovu žice (wire)
 - Enkriptovani operandi se zovu labele (lable)
- Sprovodi se u dve faze
 - Prva faza
 - A generiše sve moguće labele
 - B nabavlja labele za svoje podatke b (A ne saznaje za podatke b)
 - A generiše tabele za Bulove operatore u funkciji f
 - Druga faza – B vrši računanje

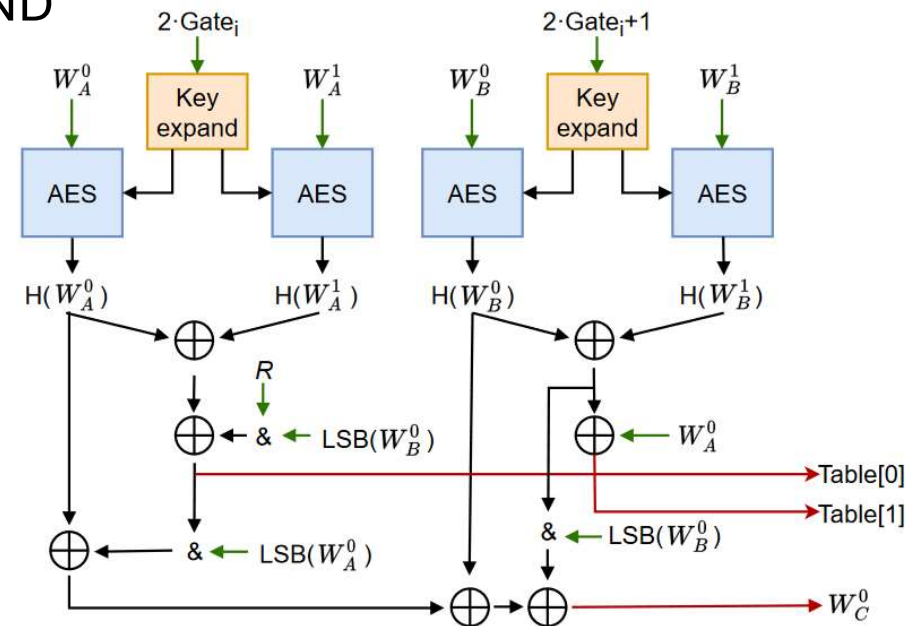


GC kola - XOR

- FreeXOR
 - Koriste se samo labele, bez tabela
 - Nema AES enkripcije
 - Protokol
 - A generiše nasumične vrednosti R i W_i^0 i postavlja W_i^1 na $W_i^0 \oplus R$
 - A podesi da važi $W_c^0 = W_a^0 \oplus W_b^0$
 - B računa $W_c = W_a \oplus W_b$

GC kola - AND

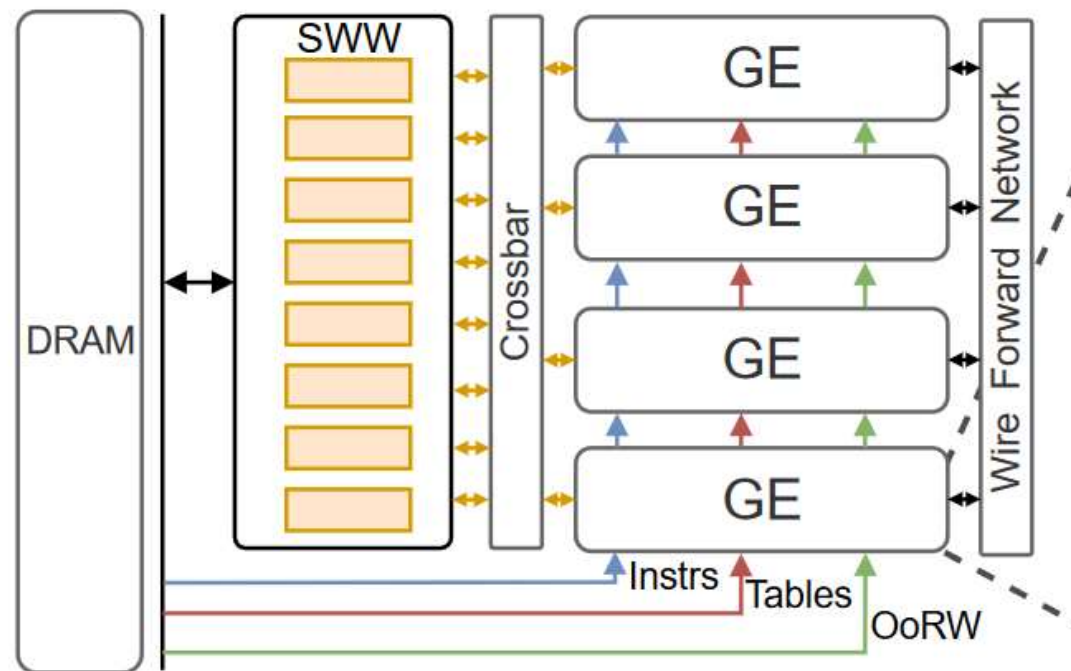
- Half-Gate
 - Optimalan način za računanje AND
 - A i B procesi su slični
- Ključevi
 - Fiksni
 - Različiti



HAAC akcelerator [1]

- Hardver uprošćen, softver mapira izračunavanja
 - Sve je poznato unapred
 - Raspored instrukcija
 - Raspored podataka
 - Transfer podataka između čipa i memorije
- Računanje i transfer podataka su preklopljeni u vremenu
 - Jedinica za izračunavanje smatra da su svi podaci spremni kad pokrene instrukciju
- Memorija na čipu
 - Random access ili streaming

Arhitektura



SWW

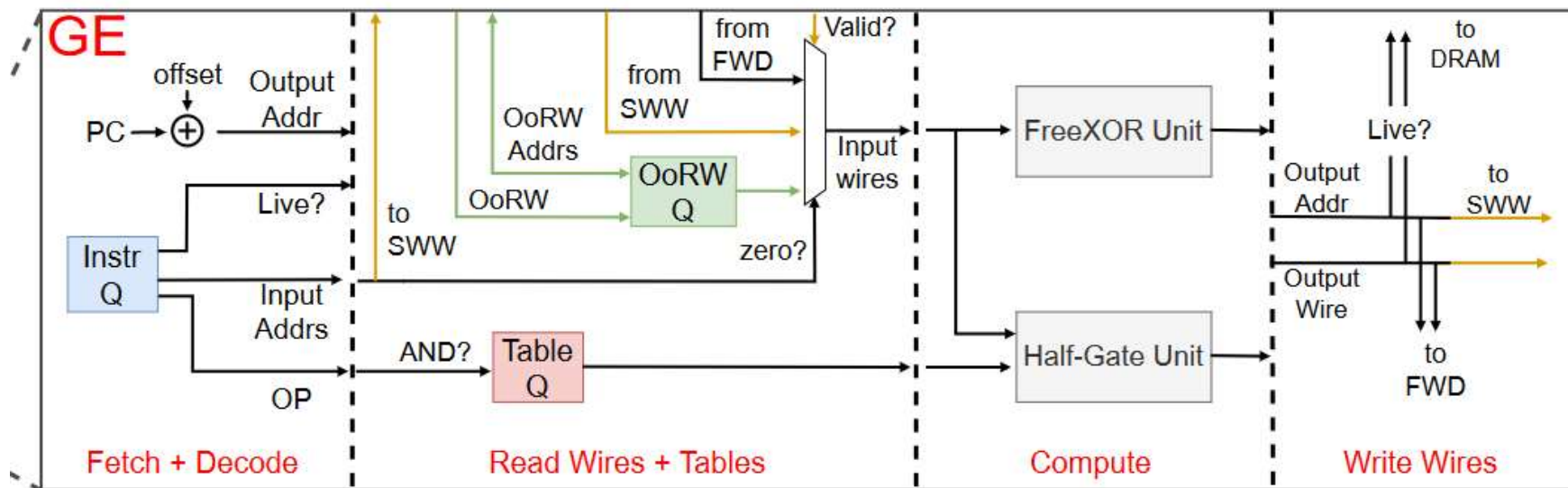
- Random access memorija
- Princip lokalnosti
 - Rezultat će se koristiti kao ulaz u skorije vreme
- Princip pomerajućih prozora
 - Nema taga za adresu
 - Adresni prostor je kontinualan
 - Prozor je vezan za adresu poslednjeg rezultata
- Memorija može da ne sadrži sve što je potrebno
- Softver radi reimenovanje adresa da bi se pogodio prozor

Streaming memorije

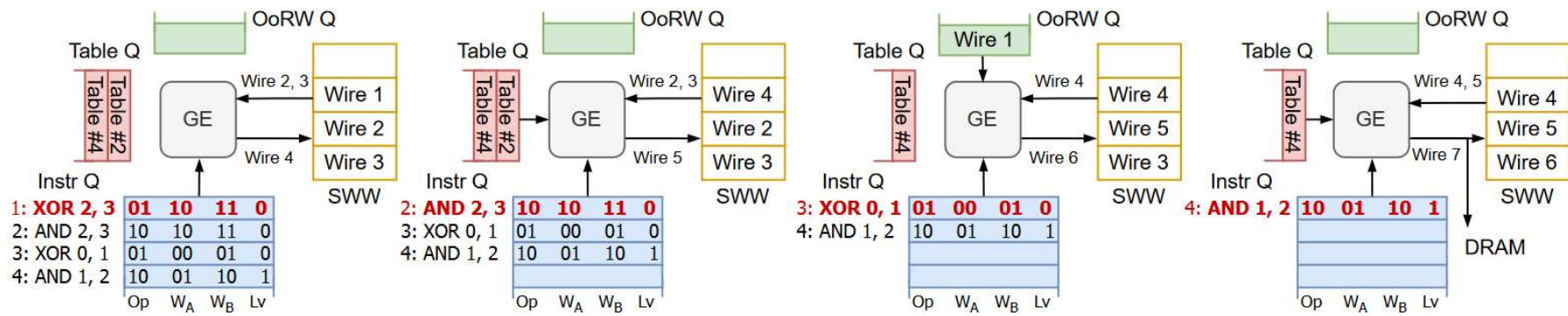
- Memorija tabela
 - Svaka tabela je vezana za instrukciju
 - Koristi se samo jednog
 - Instrukcije se izvršavaju u zadatom redosledu i to samo jednom
- Memorija za instrukcije
 - Nema kontrole toka
 - Kontrola toka se radi u samom izračunjavanju
 - Instrukcijska reč OP(2), ADR1 i ADR2 (34) i Live(1)
- Memorija za operande van prozora

GE

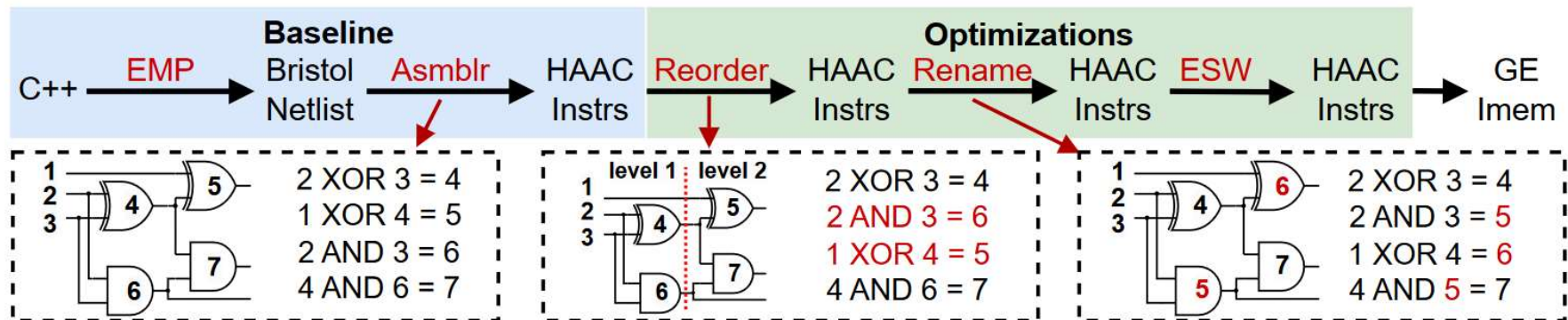
- Protočna obrada
 - Half-Gate duboka protočna obrada, FreeXor jedan ciklus



Primer izračunavanja



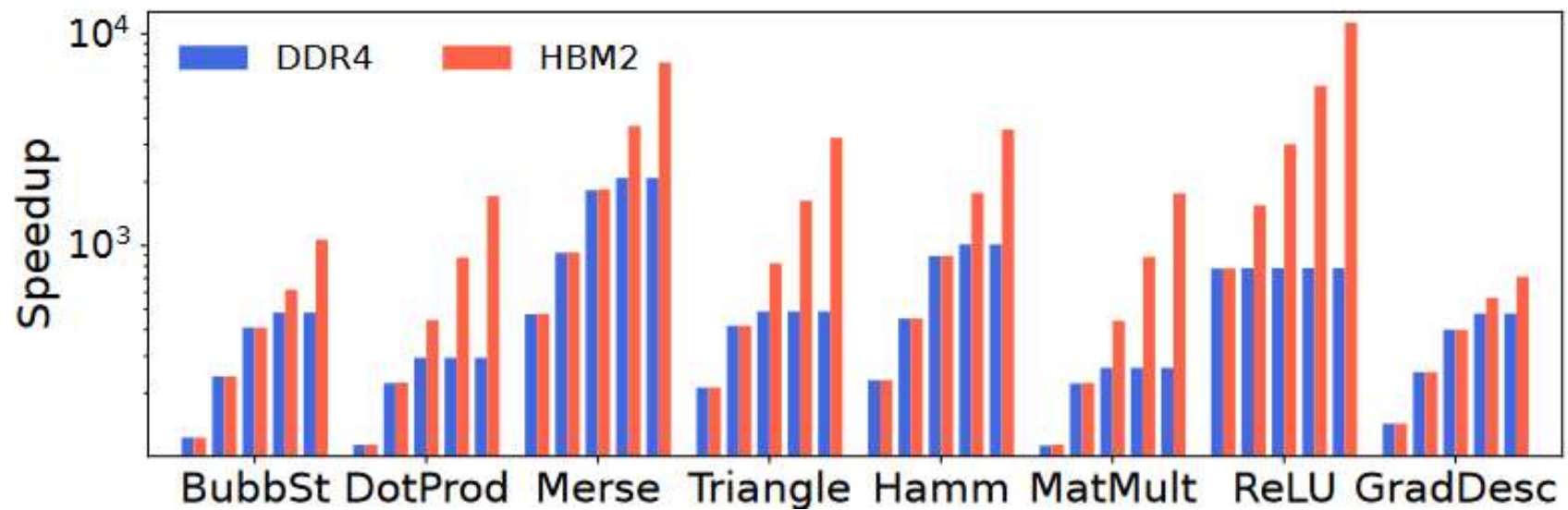
Prevodenje



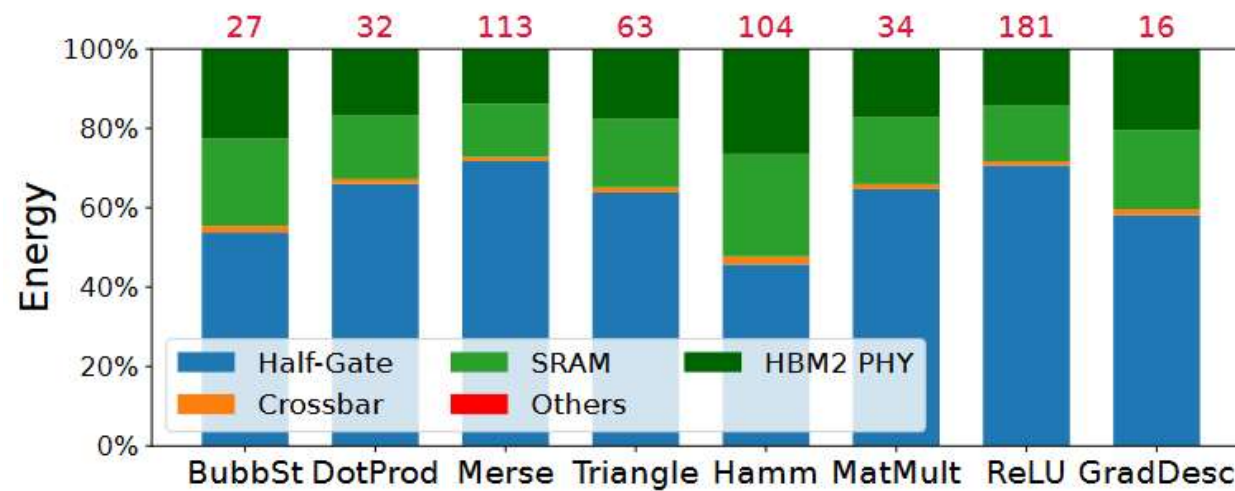
Optimizacije

- Iskorišćenje ILP promenom redosleda
 - Visok nivo ILP
 - Zavisne instrukcije su blizu da bi se povećala lokalnost podataka
 - Ideja razmaknuti zavisne instrukcije
 - Na nivou celog programa – loše
 - Na nivou dela koji može da stane u SWW
- Linerizacija rezultujućih žica
- Eliminacija potrošenih žica

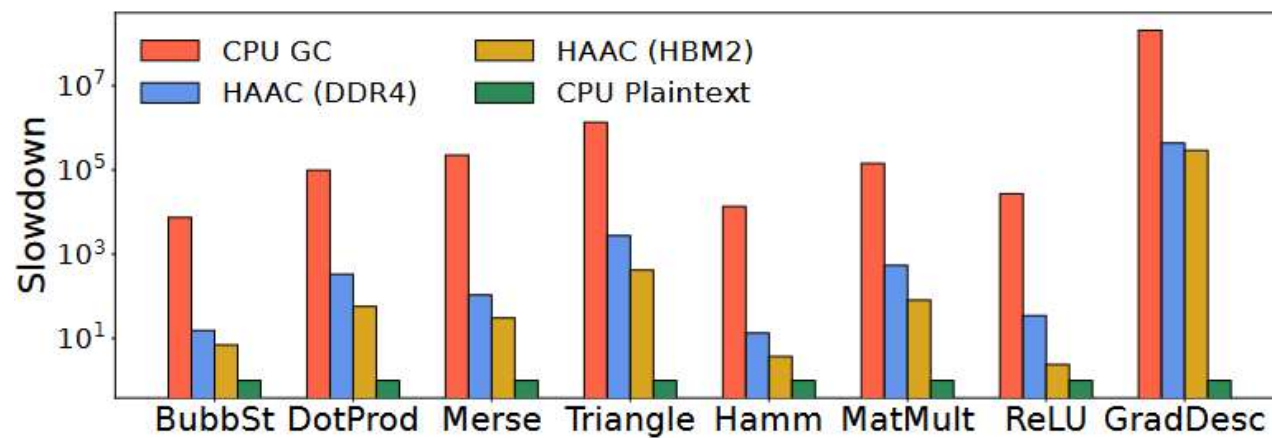
Performanse



Energija

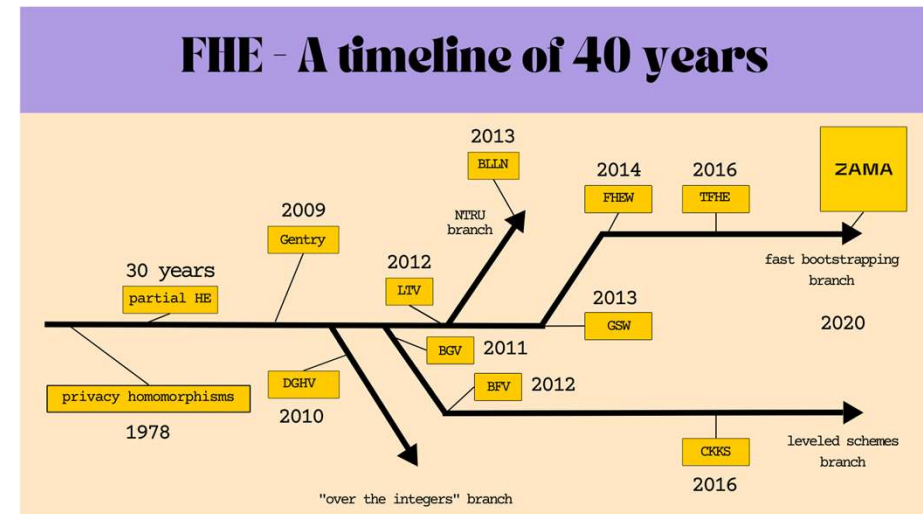


Bez privatnosti



FHE [2]

- Veliki broj šema za izračunavanje
 - Različito enkriptuju podatke
 - Tip enkriptovanih podataka isti
 - polinomi sa celobrojnim koeficijentima sa modulom Q
- Operacije sabiranja, množenja i permutacija
- Osnovne karakteristike
 - Komplekse operacije nad velikim vektorima
 - Statičko izračunavanje – podaci i operacije poznati unapred
 - Velika količina dodatnih podataka



FHE protokol

- Podaci se prebacuju u polinome
 - Sadrži tajni ključ i šum
- Sabiranje se svodi na sabiranje polinoma
- Množenje
 - Množenje i sabiranje polinoma
 - Promena tajnog ključa
 - Povećava šum
- Permutacije
 - Specijalne permutacije koeficijena
 - Promena tajnog ključa

$$c_1 = (a_1, m_1 + a_1 \cdot s + 2e_1), c_2 = (a_2, m_2 + a_2 \cdot s + 2e_2)$$

Addition:

$$c_1 + c_2 = (a_1 + a_2, (m_1 + m_2) + (a_1 + a_2) \cdot s + 2(e_1 + e_2))$$

Multiplication (BV):

$$\begin{aligned}(b_1 - a_1 \cdot s)(b_2 - a_2 \cdot s) &= (m_1 + 2e_1)(m_2 + 2e_2) \\ &= m_1 m_2 + 2(m_1 e_2 + m_2 e_1 + 2e_1 e_2)\end{aligned}$$

$$(b_1 - a_1 \cdot s)(b_2 - a_2 \cdot s) = b_1 b_2 - (b_1 a_2 + b_2 a_1)s + a_1 a_2 s^2$$

New ciphertext: $(a_1 a_2, b_1 a_2 + b_2 a_1, b_1 b_2)$ now 3 elements!

Optimizacije

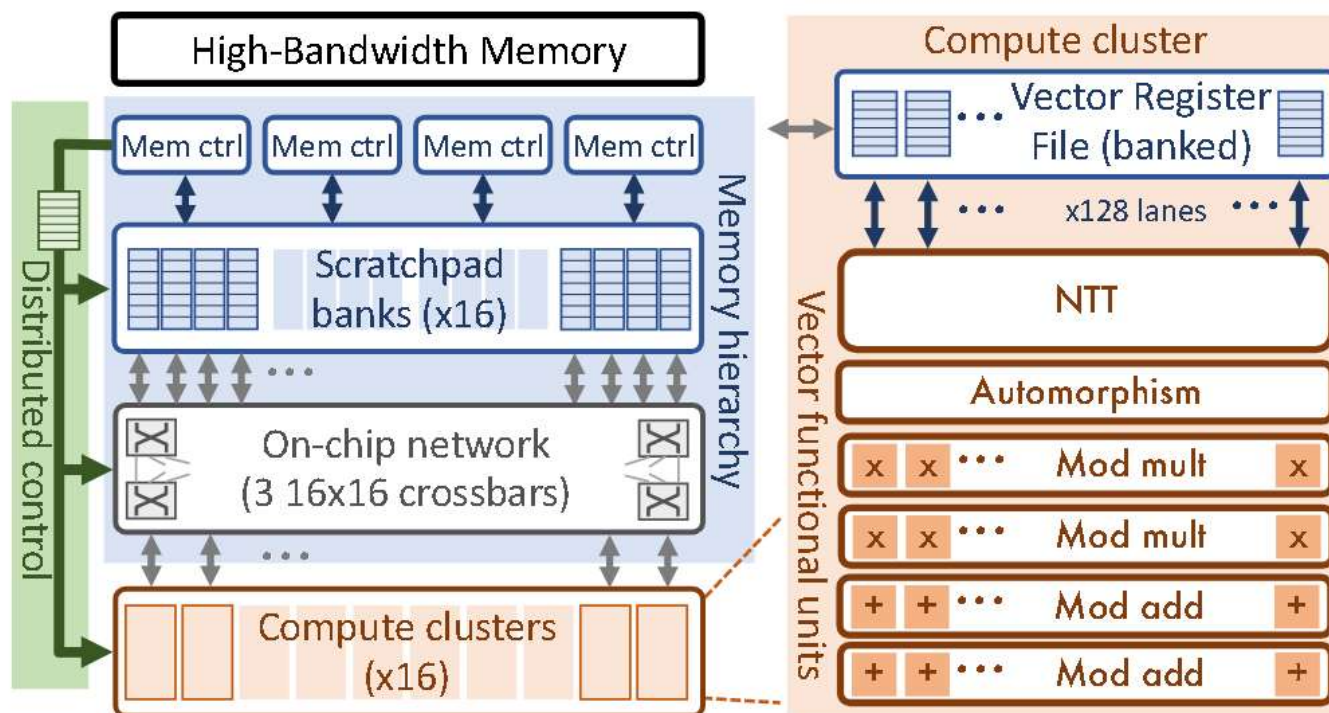
- Number-Theoretic Transform (NTT)
 - Omogućava množenje element po element
 - Ostale operacije su linearne
- Smanjenje šuma
 - Više množenja zahteva veći moduo Q
 - Bootstrapping – uklanjanje šuma delimičnom dekripcijom
 - Zahtevna operacija
 - Promena modula – nakon svakog množenja smanjiti moduo
- Residue Number System (RNS)
 - Polinom može da se predstavi polinomima sa manjim modulima

Analiza izvršavanja

```
1 def keySwitch(x: RVec[L],  
2   ksh0: RVec[L][L], ksh1: RVec[L][L]):  
3   y = [INTT(x[i], qi) for i in range(L)]  
4   u0: RVec[L] = [0, ...]  
5   u1: RVec[L] = [0, ...]  
6   for i in range(L):  
7     for j in range(L):  
8       xqj = (i == j) ? x[i] : NTT(y[i], qj)  
9       u0[j] += xqj * ksh0[i, j] mod qj  
10      u1[j] += xqj * ksh1[i, j] mod qj  
11   return (u0, u1)
```

- Promena tajnog ključa dominantna operacija
 - L^2 NTT-ijeva, $2L^2$ množenja i sabiranja vektora dužine N
 - Ostatak množenja $4L$ množenja i $3L$ sabiranja
- Dominantan transfer podataka sa i na čip
 - Treba dobra memorijska hijerarhija
 - Transfer preklopiti sa izračunavanjem
- Postoje razne varijante računanja
 - Funkcionalne jedinice treba da podržavaju samo primitivne operacije

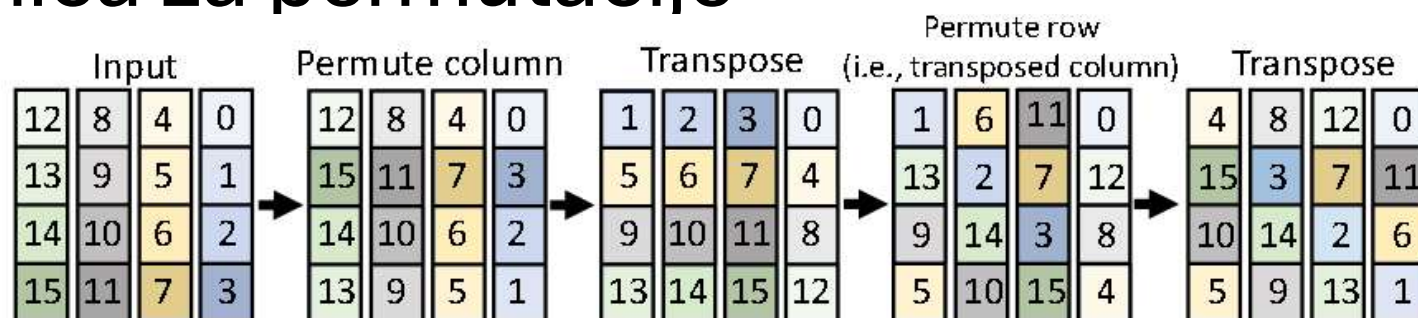
Arhitektura



Izvršavanje

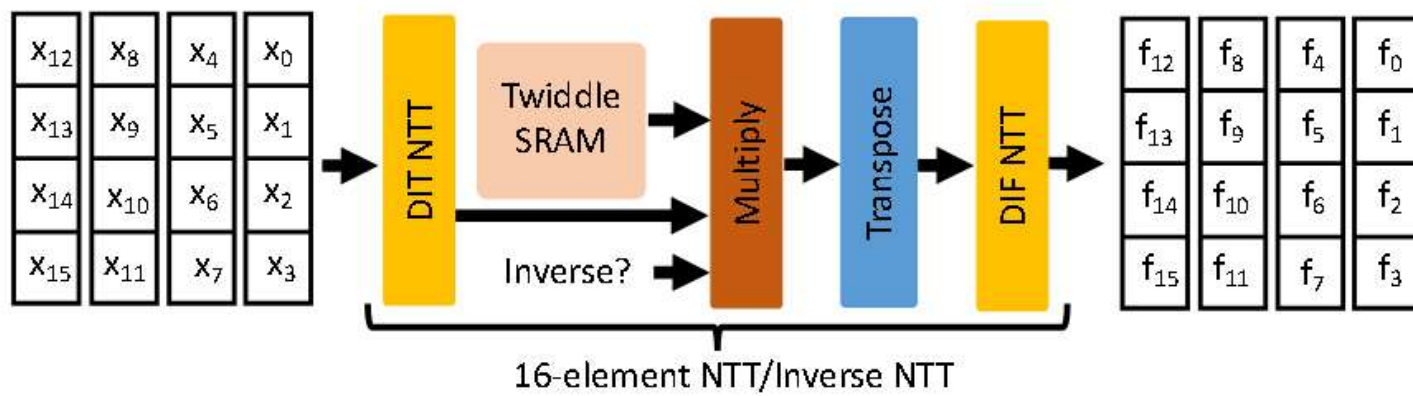
- Statički raspored instrukcija
 - Sve jedinice imaju fiksnu latenciju
- Svaka jedinica ima svoje instrukcije
 - Instrukcija ima samo jednu operaciju
- Sve je prilagođeno velikim vektorima+
- Memorija
 - Eksplicitno se kontroliše
 - Imaju puno banki
- Registarski fajl
 - Imaju banke
 - Ciklično jedinice za računanje ga koriste
- Jedinica za računanje
 - NTT, sabiranje i oduzimanje, permutacije

Jedinica za permutacije

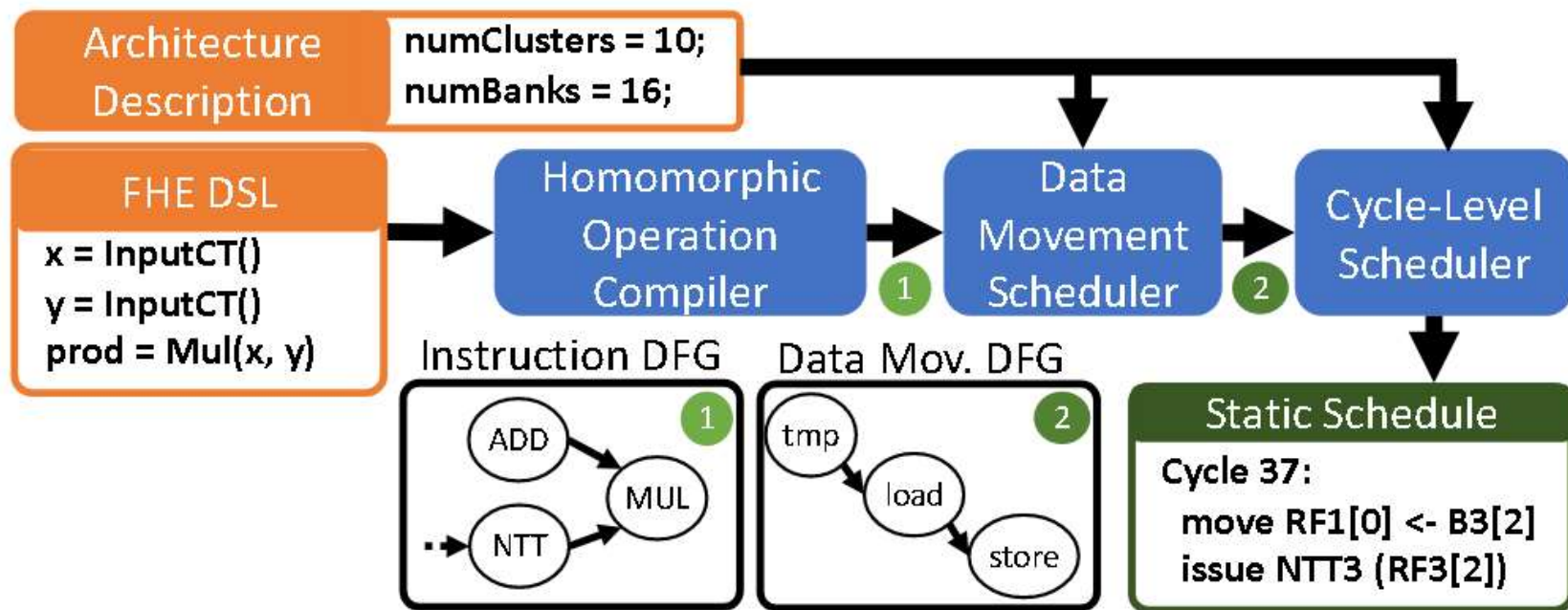


$$\left[\begin{array}{c|c} A & B \\ \hline C & D \end{array} \right]^T = \left[\begin{array}{c|c} A^T & C^T \\ \hline B^T & D^T \end{array} \right]$$

NTT

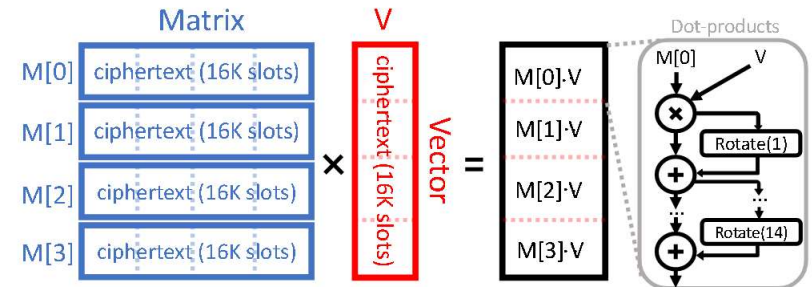


Kompajliranje (1)



Kompajliranje (2)

- Generisanje grafa izvršavanja instrukcija
 - Redosled da se smanji transfer podataka
 - Matrice za promenu ključeva kritične
 - Svaka operacija se prevodi u niz osnovnih instrukcija
- Pravljenje grafa toka podataka
 - Gramzivo rasporedjivanje
 - Teži se da se zamene mrtve vrednosti
 - Zamena radi nad onom vrednošću koja će najkasnije biti korišćenja
- Generisanje grafa sa preciznošću na nivou ciklusa
 - Proračunava se koliko unapred je potrebno dovući podatke
 - Proračunava se kada će podaci otići u memoriju



```

1 p = Program(N = 16384)
2 M_rows = [ p.Input(L = 16) for i in range(4) ]
3 output = [ None for i in range(4) ]
4 V = p.Input(L = 16)
5
6 def innerSum(X):
7     for i in range(log2(p.N)):
8         X = Add(X, Rotate(X, 1 << i))
9     return X
10
11 for i in range(4):
12     prod = Mul(M_rows[i], V)
13     output[i] = innerSum(prod)

```

Performanse

Execution time (ms) on	CPU	F1	Speedup
LoLa-CIFAR Unencryp. Wghts.	1.2×10^6	241	5,011×
LoLa-MNIST Unencryp. Wghts.	2,960	0.17	17,412×
LoLa-MNIST Encryp. Wghts.	5,431	0.36	15,086×
Logistic Regression	8,300	1.15	7,217×
DB Lookup	29,300	4.36	6,722×
BGV Bootstrapping	4,390	2.40	1,830×
CKKS Bootstrapping	1,554	1.30	1,195×
gmean speedup			5,432×

Reference

- [1] Jianqiao Mo et al., "HAAC: A Hardware-Software Co-Design to Accelerate Garbled Circuits ," ISCA, 2023
- [2] Axel Feldmann et al., "F1: A Fast and Programmable Accelerator for Fully Homomorphic Encryption," MICRO, 2021